GUÍA DE ESTUDIO

Del JSON a Power BI

Análisis de datos futbolísticos con Python y Pandas

Proceso explicado bloque por bloque, conceptos de programación y patrón de trabajo reutilizable para otros conjuntos de datos.

Proyecto práctico: Real Betis – Barcelona (7 de febrero de 2021)

Archivo de origen: `3773631.json`

Resultado: CSV preparados para Power BI.

Cómo utilizar esta guía

Esta guía sigue el proceso construido en el proyecto. Primero explica la lógica general; después, cada bloque de código, sus funciones y resultados; al final, propone un patrón reutilizable para otros datos.

Nota: se documenta la lógica del script trabajado en la conversación. La versión ejecutable es `src/prepare_match.py`.

Índice

• 1. Lógica general (ETL)

• 2. Estructura del proyecto

• 3. Imports y rutas

• 4. Carga JSON y DataFrame

• 5. Diagnóstico inicial

• 6. Auditoría de eventos

• 7. Filtrado de tiros y xG

• 8. Validación de goles

• 9. Coordenadas

• 10. Selección y exportación

• 11. Errores y depuración

• 12. Patrón reutilizable

• 13. Checklist y glosario

1. Objetivo y lógica general

Hemos creado un proceso sencillo de preparación de datos: leer un JSON, convertirlo en tabla, inspeccionar el contenido, seleccionar información útil y exportar CSV que Power BI pueda consumir.

Fase	Significado	En este proyecto
Extract — extraer	Leer los datos de origen.	Abrimos 3773631.json y cargamos los eventos.
Transform — transformar	Cambiar estructura y seleccionar información.	Aplanamos el JSON, auditamos eventos, filtramos tiros y extraemos coordenadas.
Load — cargar/exportar	Guardar datos para la siguiente herramienta.	Exportamos eventos_partido.csv y tiros_partido.csv.

```
3773631.json
   |
   v
events = json.load(file)
   |
   v
df = pd.json_normalize(events)    4.004 filas x 113 columnas
   |
   +--> events_export --> eventos_partido.csv
   |
   +--> shots (solo Shot) --> shots_export --> tiros_partido.csv
```

Importante: 4.004 filas son eventos de varios tipos, no 4.004 tiros. La tabla de tiros contiene 20 filas porque 20 eventos están clasificados como Shot.

2. Estructura del proyecto

```
DATOS FUTBOL/
├── data/
│   ├── raw/3773631.json
│   └── processed/
│       ├── eventos_partido.csv
│       └── tiros_partido.csv
├── src/prepare_match.py
├── reports/
├── README.md
├── .gitignore
└── analisis.ipynb   (opcional)
```

raw contiene el origen sin modificar; processed contiene archivos derivados; src contiene el código; reports queda para informes. La separación ayuda a reproducir el proceso.

3. Bloque 1: importar bibliotecas

```
import json
from pathlib import Path
import pandas as pd
```

Línea	Explicación
import json	Módulo estándar para leer datos JSON.
from pathlib import Path	Herramienta para construir y manipular rutas.
import pandas as pd	Importa Pandas con el alias pd.

Una biblioteca es código reutilizable. Importarla permite usar sus funciones, pero no carga todavía los datos.

4. Bloque 2: localizar el archivo

```
PROJECT_DIR = Path(__file__).resolve().parent.parent
JSON_PATH = PROJECT_DIR / "data" / "raw" / "3773631.json"
```

Elemento	Significado
__file__	Ruta del archivo Python que se ejecuta.
.resolve()	Obtiene una ruta absoluta normalizada.
.parent	Devuelve la carpeta contenedora. Dos parent suben desde src hasta la raíz.
Path / 'carpeta'	Une partes de una ruta; aquí / no es división matemática.
PROJECT_DIR / JSON_PATH	Variables con la raíz del proyecto y la ruta del JSON.

Ventaja: si mueves el proyecto y conservas la estructura, el código puede seguir encontrando el archivo sin escribir una ruta absoluta fija.

5. Bloque 3: leer JSON y crear el DataFrame

```
with open(JSON_PATH, "r", encoding="utf-8") as file:
    events = json.load(file)

df = pd.json_normalize(events)
```

Leer el archivo

• open(JSON_PATH, 'r') abre el archivo en modo lectura.

• encoding='utf-8' ayuda a leer tildes, eñes y caracteres especiales.

• as file da un nombre al archivo abierto.

• with gestiona el cierre del archivo al terminar el bloque.

• json.load(file) convierte el contenido JSON en estructuras de Python.

Aplanar la estructura

pd.json_normalize(events) convierte estructuras anidadas en columnas. Por ejemplo, un objeto team con name puede convertirse en team.name; un objeto shot con outcome.name puede convertirse en shot.outcome.name.

```
# Ejemplo ilustrativo de un evento
{
  "minute": 37,
  "team": {"name": "Real Betis"},
  "shot": {
    "statsbomb_xg": 0.440972,
    "outcome": {"name": "Goal"}
  }
}
```

Después de normalizar: minute, team.name, shot.statsbomb_xg y shot.outcome.name quedan accesibles como columnas. El DataFrame df tiene 4.004 filas y 113 columnas.

6. Bloque 4: diagnóstico inicial

```
print(df.shape[0])  # filas
print(df.shape[1])  # columnas
print(df[["minute", "type.name", "team.name", "player.name"]].head())
print(df.dtypes)
print(df.isna().sum().sort_values(ascending=False).head(15))
```

Expresión	Función
df.shape	Devuelve (filas, columnas).
shape[0] / shape[1]	Acceden a filas / columnas; los índices empiezan en cero.
df[[...]]	Selecciona varias columnas.
head()	Muestra las primeras cinco filas por defecto.
dtypes	Muestra tipos de datos.
isna()	Marca los valores ausentes.
sum()	Cuenta los ausentes por columna en este contexto.
sort_values(ascending=False)	Ordena de mayor a menor.
head(15)	Muestra las quince columnas con más nulos.

Muchos nulos no significan automáticamente un error. Un campo de xG solo corresponde a tiros; un campo de faltas solo puede aplicar a eventos de falta. Hay que entender el significado antes de borrar datos.

7. Bloque 5: auditar los eventos

```
df["type.name"].value_counts(dropna=False)
df["team.name"].value_counts(dropna=False)
```

value_counts cuenta la frecuencia de cada valor. dropna=False incluye los ausentes. to_string() se usa en el script para imprimir una representación legible.

Resultado	Interpretación
Pass: 1.184	Eventos de pase registrados.
Carry: 938	Eventos de conducción.
Shot: 20	Eventos clasificados como tiro.
Barcelona: 2.449 eventos	Eventos asociados al Barcelona.
Real Betis: 1.555 eventos	Eventos asociados al Betis.

El número de eventos no demuestra por sí solo qué equipo jugó mejor. Es un recuento, no una evaluación completa del rendimiento.

8. Bloque 6: filtrar tiros y calcular xG

```
shots = df[df["type.name"] == "Shot"].copy()
```

La condición crea valores True/False. df[...] conserva filas donde la condición es verdadera. copy() crea una copia independiente del subconjunto. Resultado: 20 filas.

```
len(shots)
shots["team.name"].value_counts()
shots["shot.outcome.name"].value_counts(dropna=False)
```

len() cuenta las filas; value_counts() cuenta las categorías. Con ello obtenemos tiros por equipo y resultados como Goal, Blocked, Saved, Off T y Wayward.

Agrupar y agregar

```
summary = shots.groupby("team.name").agg(
    tiros=("id", "count"),
    goles_en_tiros=(
        "shot.outcome.name",
        lambda s: (s == "Goal").sum(),
    ),
    xG=("shot.statsbomb_xg", "sum"),
)
print(summary.round(3).to_string())
```

Elemento	Explicación
groupby('team.name')	Divide las filas en grupos por equipo.
agg(...)	Calcula varias métricas por grupo.
('id', 'count')	Cuenta identificadores de tiro.
lambda s: (s == 'Goal').sum()	Cuenta los resultados Goal; True suma como 1.
('shot.statsbomb_xg', 'sum')	Suma el xG de los tiros.
round(3)	Redondea valores a tres decimales.

Equipo	Tiros	Goles en tiros	xG
Barcelona	12	2	0,988
Real Betis	8	2	1,395

xG significa goles esperados. Un tiro con xG 0,40 tiene una probabilidad estimada de gol del 40 % según el modelo. Sumar xG ofrece una estimación acumulada de la calidad de las ocasiones, no una predicción exacta del marcador.

9. Bloque 7: validar goles

```
goals = shots[shots["shot.outcome.name"] == "Goal"]

own_goals = df[
    df["type.name"].isin(["Own Goal For", "Own Goal Against"])
]

if own_goals.empty:
    print("No se han encontrado eventos de gol propio.")
else:
    print(own_goals[columns_to_show].to_string(index=False))
```

Elemento	Explicación
goals = ...	Filtra tiros con resultado Goal.
isin([...])	Comprueba si un valor pertenece a una lista.
empty	Indica si el DataFrame no contiene filas.
if / else	Elige qué bloque ejecutar según una condición.
columns_to_show	Lista de columnas que se mostrarán.

El archivo tiene Own Goal For para Barcelona y Own Goal Against para Betis en el minuto 67. Son dos registros del mismo gol en propia puerta, no dos goles. El marcador 2–3 combina dos goles del Betis, dos goles del Barcelona procedentes de tiros y el gol en propia puerta a favor del Barcelona.

10. Bloque 8: extraer coordenadas

```
def extract_coordinates(series, axis):
    return series.apply(
        lambda value: (
            value[axis]
            if isinstance(value, list) and len(value) > axis
            else None
        )
    )
```

Elemento	Explicación
def	Define una función reutilizable.
series	Columna cuyos valores contienen listas de coordenadas.
axis	Índice: 0 para x y 1 para y.
apply()	Aplica la operación a cada elemento de la Serie.
isinstance(value, list)	Comprueba que el valor sea una lista.
len(value) > axis	Comprueba que existe el elemento pedido.
value[axis]	Extrae la coordenada.

Elemento	Explicación
else None	Devuelve un valor ausente si no hay coordenada válida.

Ejemplo: [115.3, 34.7] da x=115.3 con axis=0 e y=34.7 con axis=1.

```
for source_column, prefix in [
    ("location", "shot"),
    ("shot.end_location", "end"),
]:
    if source_column in shots_export.columns:
        shots_export[f"{prefix}_x"] = extract_coordinates(
            shots_export[source_column], 0
        )
        shots_export[f"{prefix}_y"] = extract_coordinates(
            shots_export[source_column], 1
        )
        shots_export = shots_export.drop(columns=[source_column])
```

El bucle procesa la posición inicial (location) y la posición final registrada (shot.end_location). Crea shot_x, shot_y, end_x y end_y; después elimina las columnas que contenían las listas originales.

Las coordenadas permiten representar los tiros en un campo, pero hay que respetar el sistema de coordenadas y revisar la orientación antes de interpretar el mapa.

11. Bloque 9: seleccionar y exportar

Crear la carpeta de salida

```
OUTPUT_DIR = PROJECT_DIR / "data" / "processed"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

mkdir crea la carpeta si no existe; parents=True crea carpetas superiores necesarias; exist_ok=True permite repetir la ejecución si la carpeta ya existe.

Seleccionar columnas

```
event_columns = [
    "id", "index", "period", "minute", "second", "timestamp",
    "type.name", "team.name", "possession_team.name",
    "player.name", "possession", "play_pattern.name",
    "location", "pass.end_location", "shot.end_location",
    "shot.statsbomb_xg", "shot.outcome.name",
]
event_columns = [col for col in event_columns if col in df.columns]
events_export = df[event_columns].copy()
```

La lista define qué campos conservar. La comprensión de lista mantiene solo columnas existentes. Se hace una copia y después se extraen coordenadas. Por eso la tabla final tiene 20 columnas, más que las 17 inicialmente seleccionadas.

Renombrar columnas

```
shots_export = shots_export.rename(columns={
    "team.name": "equipo",
    "player.name": "jugador",
    "shot.statsbomb_xg": "xg",
    "shot.outcome.name": "resultado",
    "minute": "minuto",
    "second": "segundo",
    "period": "periodo",
})
```

rename cambia los nombres de las columnas, no los valores.

Exportar CSV

```
events_export.to_csv(events_path, index=False, encoding="utf-8-sig")
shots_export.to_csv(shots_path, index=False, encoding="utf-8-sig")
```

Argumento	Función
to_csv()	Guarda el DataFrame en formato CSV.
events_path / shots_path	Rutas de destino.
index=False	No añade el índice de Pandas como columna extra.
encoding='utf-8-sig'	Ayuda a Excel a reconocer caracteres especiales.

Resultados: eventos_partido.csv tiene 4.004 filas y 20 columnas; tiros_partido.csv tiene 20 filas y 18 columnas.

12. Errores y depuración

Error / situación	Causa probable	Qué comprobar
NameError con own_goals	Variable no definida antes de usarla.	Buscar la definición y el orden de ejecución.
KeyError con shot_x / shot_y	Los nombres esperados no coinciden con las columnas reales.	Imprimir shots_export.columns.tolist().
CSV creado, pero el script falla después	El error ocurre después de exportar, por ejemplo en la vista previa.	Leer la última línea del traceback.
Archivo no visible en VS Code	Ruta incorrecta o explorador sin actualizar.	Revisar la ruta impresa y actualizar el explorador.
Muchos valores nulos	El atributo solo aplica a ciertos eventos.	Examinar el significado y el tipo de evento antes de eliminar.

Método de depuración: lee la última línea del traceback; identifica la variable o columna; imprime la estructura real con type(), .columns o .head(); corrige la causa y vuelve a ejecutar.

13. Patrón reutilizable para otros datos

No empieces directamente con los gráficos. Sigue este orden, documenta las decisiones y guarda las comprobaciones.

Paso 1 — Definir la pregunta

¿Qué quiero averiguar? ¿Qué representa una fila? ¿Qué conclusión debería apoyar el análisis?

Paso 2 — Identificar el origen

Anota formato, ruta, fuente, fecha de extracción, licencia y limitaciones conocidas.

Paso 3 — Organizar carpetas

Separa origen, datos procesados, código, informes y documentación.

Paso 4 — Cargar

Usa la biblioteca adecuada y comprueba que el archivo se abrió correctamente.

Paso 5 — Perfilar

Revisa filas, columnas, tipos, nulos, valores únicos, duplicados, rangos y ejemplos.

Paso 6 — Validar el significado

Comprueba qué representa cada fila, las claves y la coherencia con cifras conocidas.

Paso 7 — Transformar con criterio

Selecciona columnas, normaliza nombres, corrige tipos, trata nulos y crea variables derivadas.

Paso 8 — Validar después

Compara conteos antes y después; revisa duplicados, rangos y totales.

Paso 9 — Exportar

Genera tablas claras con nombres consistentes y rutas reproducibles.

Paso 10 — Visualizar

Importa a Power BI y diseña visualizaciones que respondan a la pregunta inicial.

Paso 11 — Documentar y versionar

Explica objetivo, fuente, cómo ejecutar, supuestos, limitaciones y hallazgos.

Plantilla mínima para un CSV

```
from pathlib import Path
import pandas as pd

PROJECT_DIR = Path(__file__).resolve().parent.parent
```

```
INPUT_PATH = PROJECT_DIR / "data" / "raw" / "archivo_origen.csv"
OUTPUT_DIR = PROJECT_DIR / "data" / "processed"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

# Cargar
df = pd.read_csv(INPUT_PATH)

# Perfilar
print("Shape:", df.shape)
print(df.head())
print(df.dtypes)
print(df.isna().sum().sort_values(ascending=False).head(10))

# Validar: claves, duplicados, rangos y reglas del dominio.

# Transformar: seleccionar columnas, corregir tipos,
# tratar ausentes y crear variables necesarias.

# Exportar
df.to_csv(OUTPUT_DIR / "datos_preparados.csv",
          index=False, encoding="utf-8-sig")
```

Esta plantilla es para un CSV. Si la fuente es JSON, Excel o SQL, cambiará el bloque de carga, pero el orden perfilado → validación → transformación → exportación sigue siendo útil.

14. Checklist de calidad

- ☐ ¿Puedo explicar qué representa una fila?
- ☐ ¿Comprobé el número de filas y columnas?
- ☐ ¿Inspeccioné tipos, nulos y valores únicos?
- ☐ ¿Comprobé claves y posibles duplicados?
- ☐ ¿Contrasté las cifras principales con referencias fiables?
- ☐ ¿Documenté por qué conservo o transformo datos?
- ☐ ¿Validé el resultado después de transformar?
- ☐ ¿Los nombres de columnas son claros y consistentes?
- ☐ ¿Los CSV se abren y se pueden reutilizar?
- ☐ ¿Otra persona podría ejecutar el proceso con el README?

Glosario rápido

Término	Definición
JSON	Formato de texto que puede contener objetos, listas y estructuras anidadas.
DataFrame	Tabla de Pandas con filas y columnas.
Serie (Series)	Estructura unidimensional de Pandas, como una columna.
Evento	Acción registrada; no equivale necesariamente a una jugada completa.
Filtrado	Selección de filas que cumplen una condición.
Agregación	Cálculo de resúmenes como conteos y sumas, normalmente por grupo.
Nulo	Valor no disponible; no significa automáticamente error.
xG	Estimación de la probabilidad de que un disparo termine en gol.
CSV	Formato de texto tabular que separa campos con delimitadores.
ETL	Extract, Transform, Load: extraer, transformar y cargar/exportar.
Reproducibilidad	Poder repetir el proceso con la misma fuente y configuración.

Resumen final

La idea clave no es memorizar cada función, sino aprender el orden: definir la pregunta, conocer la fuente, perfilar los datos, validar su significado, transformar con criterio, comprobar el resultado y solo entonces visualizar.

En este proyecto: el JSON se convirtió en un DataFrame de 4.004 eventos; filtramos 20 tiros; revisamos los goles y el xG; separamos coordenadas y exportamos dos CSV. El siguiente paso es revisar los archivos y después importarlos a Power BI.

Fin de la guía.